KU LEUVEN

FACULTEIT INDUSTRIËLE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Labo 2

Labo Geavanceerde Computerarchitectuur

Kjell NAESSENS
Beau FORREZ

Labo verantwoordelijke: T. Feys

Academiejaar 2025-2026

1 Context

In deze opdracht wordt het probleem van het vinden van het maximum van een array onderzocht met drie verschillende benaderingen: een CPU for-loop en twee GPU-implementaties: GPU kernel met atomic operation en GPU kernel met reduction. Het doel is niet alleen om correcte resultaten te verkrijgen, maar ook om het effect van de array-grootte op de uitvoeringstijd te analyseren.

2 Methodologie

2.1 CPU for-loop

In deze eerste implementatie wordt het maximum van een array bepaald met een eenvoudige for-loop op de CPU. De array wordt gevuld met willekeurige gehele getallen tussen 0 en N-1. Vervolgens wordt door elk element van de array gelopen en wordt telkens gecontroleerd of het huidige element groter is dan de tot dan toe gevonden maximumwaarde. Indien dat zo is, wordt deze waarde als nieuwe maximum opgeslagen. De uitvoering gebeurt sequentieel: elk element wordt één voor één vergeleken, zonder enige vorm van parallellisatie. Dit maakt de methode geschikt voor kleine datasets, maar minder efficiënt voor grote arrays. De tijd die de operatie nodig had, wordt ook geprint.

De code in Listing 1 toont de implementatie van deze CPU-versie.

2.2 GPU kernel met atomic operation

Deze implementatie bepaalt het maximum van een array met een CUDA-kernel op de GPU. Elke thread verwerkt één element en gebruikt atomicMax om het globale maximum veilig bij te werken zonder race conditions.

De array wordt eerst naar het GPU-geheugen gekopieerd, waarna de kernel met één block van N threads wordt uitgevoerd. De uitvoeringstijd wordt gemeten met CUDA-events en weergegeven in microseconden. Deze parallelle aanpak kan bij grotere arrays veel sneller zijn dan de sequentiële CPU-versie, hoewel het mechanisme dat race conditions voorkomt nog steeds voor heel wat vertraging zorgt.

De code in Listing 2 toont de implementatie van deze GPU-versie.

2.3 GPU kernel met reduction

In deze derde implementatie wordt het maximum van de array bepaald met behulp van een reductietechniek op de GPU. In plaats van elke thread afzonderlijk het globale maximum te laten zoeken via een atomicMax-operatie, werken de threads hier samen binnen een block om lokaal het maximum te berekenen. Elke thread vergelijkt waarden in een gedeeld geheugen en reduceert deze tot één maximum per block. Vervolgens worden de resultaten van alle blocks gecombineerd om het

uiteindelijke maximum te vinden. Dit vermindert het aantal globale geheugentoegangen en maakt de uitvoering efficiënter, zeker bij grote datasets.

De array wordt opnieuw naar het GPU-geheugen gekopieerd, waarna de kernel met meerdere blocks en threads wordt uitgevoerd. De uitvoeringstijd wordt gemeten met CUDA-events, zoals bij de vorige implementatie. Deze benadering benut de parallelle structuur van de GPU beter dan de versie met atomicMax, en resulteert doorgaans in kortere rekentijden bij grote arrays.

De code in Listing 3 toont de implementatie van deze *reduction*-gebaseerde GPU-versie.

3 Observaties en resultaten

Tabel 1 Uitvoeringstijd van verschillende implementaties voor verschillende arraygroottes

Array grootte N	CPU for-loop (ns)	GPU atomic operation (µs)	GPU reduction (µs)
128	79	7.2	50
256	165	7.2	50
512	330	7.2	50
1024	645	7.2	50
8192	4722	7.2	50
65536	37077	7.2	50

3.1 Vragen

3.2 Is het resultaat correct?

Ja, het resultaat is correct zolang de kernel en geheugenoperaties correct worden uitgevoerd. Zowel de CPU- als GPU-implementaties leveren hetzelfde maximum, wat kan worden gecontroleerd door de resultaten met elkaar te vergelijken of handmatig het maximum te zoeken in de geprinte array.

3.3 Hoe neemt de tijd toe in functie van de inputgrootte

De uitvoeringstijd neemt lineair toe met de grootte van de input voor de CPU-versie, omdat elk element sequentieel wordt vergeleken. Bij de GPU-kernels blijft de uitvoeringstijd gelijk, aangezien meerdere elementen gelijktijdig worden verwerkt. Enkel bij kleine arrays kan de GPU trager zijn door de overhead van geheugenoverdracht.

3.4 Wat gebeurt er als de input groter is dan 2048 (hoe wordt een correcte berekening gegarandeerd en zijn er aanvullende synchronisatiestappen)?

Als de input groter is dan 2048, volstaat één block niet meer. Er moeten meerdere blocks gebruikt worden. Elk block berekent dan een lokaal maximum, waarna een tweede kernel of een CPU-stap deze partiële maxima combineert. Extra synchronisatie is nodig tussen blocks om het globale maximum correct te berekenen.

3.5 Overweeg om de kernel op te roepen met veel minder threads dan er gegevenselementen zijn. Bijvoorbeeld een tiende. Hoe zorg je er voor dat alle elementen worden verwerkt?

In dat geval kan elke thread meerdere elementen verwerken in een lus. Dit gebeurt door de index te verhogen met de totale gridgrootte (idx += blockDim.x * gridDim.x) zodat alle elementen van de array uiteindelijk aan bod komen.

3.6 Hoe pak je een 2D-array aan?

Een 2D-array kan worden aangepakt door de indices om te rekenen naar één dimensie, bijvoorbeeld met idx = x + y * width. Alternatief kan een 2D-grid en 2D-blockindeling worden gebruikt, waarbij threadIdx.x en threadIdx.y rechtstreeks overeenkomen met de rijen en kolommen van de array.

4 Besluit

We vergeleken de uitvoering van sequentiële en parallelle operaties op een array. Bij de sequentiële operatie (op de CPU) zagen we een lineair verband in uitvoeringstijd en array-grootte. Bij de parallelle uitvoering (op de GPU) zagen we een constante uitvoeringstijd, onafhankelijk van de array-grootte.

We verwachtten dat de reduction-methode goedkoper (in tijd) zou zijn dan de atomic-methode, maar bekwamen het omgekeerde resultaat. Hierdoor vermoeden we dat er nog ergens overhead in onze code aanwezig is. We zien duidelijk dat er vanaf een bepaalde array-grootte beter gekozen wordt voor parallelle uitvoering, zoals gedemonstreerd in tabel 1.

5 Appendix

5.1 CPU nester for-loop

```

#include <iostream>
#include <cstdlib> // for rand() and srand()
#include <chrono> // for timing
#include <ctime> // for random seed

int main() {
    const int N = 1000; // length of the array
    const int runs = 10000; // number of repeats
    int a[N];

    // Random number generation
    srand(static_cast<unsigned>(time(nullptr)));
    for (int i = 0; i < N; ++i) a[i] = rand() % 100;

    // Start timing
    int result = 0;
    auto t0 = std::chrono::high_resolution_clock::now();

    // Repeatedly find the maximum
    for (int r = 0; r < runs; ++r) {
        int m = a[0];
        for (int i = 1; i < N; ++i) {
            if (a[i] > m) m = a[i]; // if current element is larger, update max
        }
        result = m; // keep last result
    }

    // End timing
    auto t1 = std::chrono::high_resolution_clock::now();

    // Calculate average duration
    long long total_ns = std::chrono::duration_cast<std::chrono::nanoseconds>(
        t1 - t0).count();
    double avg_ns = static_cast<double>(total_ns) / runs;

    // Print Input
    std::cout << "Input array: ";
    for (int i = 0; i < N; ++i) {
        std::cout << a[i] << " ";
    }
    std::cout << "\n";

    // Print Result
    std::cout << "Result: " << result << "\n";
    std::cout << "Average per run: " << avg_ns << " ns\n";
```

```
    return 0;
}
```

Listing 1: C++ code CPU

5.2 GPU kernel met atomic operation

```

#include <iostream>
#include <cuda_runtime.h>
#include <cstdlib> // for random numbers
#include <ctime>   // for random seed

// timing the GPU
cudaEvent_t start_cuda, stop_cuda;

// CUDA kernel to find the max of an array using atomic operation
__global__ void max(int *a, int *b, int N) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < N) {
        atomicMax(b, a[idx]); // find max between b and a[idx]
    }
}

int main() {
    const int N = 10; // length of array
    int a[N], b; // input array, output value
    int *d_a, *d_b; // device pointers

    // Create timing events
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);

    // Seed random number generator
    srand(static_cast<unsigned>(time(nullptr)));

    // Fill array with random numbers 0-99
    for (int i = 0; i < N; i++) {
        a[i] = rand() % 100;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, sizeof(int));

    // Copy input array to device
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    // Initialize d_b to 0 on device before kernel launch
    int initial_max = 0;
```

```
    cudaMemcpy(d_b, &initial_max, sizeof(int), cudaMemcpyHostToDevice);

    // Start timing
    cudaEventRecord(start_cuda);

    // Launch kernel to find max
    max<<<1, N>>>(d_a, d_b, N);

    // Wait for kernel to finish
    cudaDeviceSynchronize();

    // Stop timing
    cudaEventRecord(stop_cuda);
    cudaEventSynchronize(stop_cuda);

    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start_cuda, stop_cuda);

    // Copy result back to host
    cudaMemcpy(&b, d_b, sizeof(int), cudaMemcpyDeviceToHost);

    // Print array and result
    std::cout << "Array: ";
    for (int i = 0; i < N; i++) {
        std::cout << a[i] << " ";
    }
    std::cout << "\n";

    std::cout << "Max value found: " << b << std::endl;
    std::cout << "Elapsed time: " << milliseconds << " ms" << std::endl;

    // Cleanup
    cudaFree(d_a);
    cudaFree(d_b);
    cudaEventDestroy(start_cuda);
    cudaEventDestroy(stop_cuda);

    return 0;
}
```

Listing 2: C++ code GPU1

5.3 GPU kernel met reduction

```

#include <iostream>
#include <cuda_runtime.h>
#include <cstdlib> // for random numbers
#include <ctime>   // for random seed

```

```cpp
// Cuda kernel to find the max of an array using reduction and shared memory
__global__ void max_reduction(int *input, int *output, int numElements) {
    extern __shared__ int sdata[]; // shared memory per block

    unsigned int tid = threadIdx.x;
    unsigned int i = blockIdx.x * (blockDim.x*2) + threadIdx.x;

    // Load first element into shared memory
    sdata[tid] = max(input[i], input[i + blockDim.x]);
    __syncthreads();

    // Do reduction in shared memory
    for (unsigned int s=blockDim.x/2; s>0; s>>=1) { // for each stride
        if (tid < s) { // for even indexed threads
            // replace if second element is larger
            if (sdata[tid + s] > sdata[tid]) {
                sdata[tid] = sdata[tid + s];
            }
        }
        __syncthreads();
    }

    // Write result for this block to global memory
    if (tid == 0) {
        output[blockIdx.x] = sdata[0];
    }
}

int main() {
    const int N = 1024; // length of the array
    const int runs = 10000; // number of repeated runs for averaging
    int a[N]; // input array
    int *d_a, *d_b; // device pointers

    // Timing events
    cudaEvent_t start_cuda, stop_cuda;
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);

    // Random number generator
    srand(static_cast<unsigned>(time(nullptr)));

    // Fill array with random numbers 0-99
    for (int i = 0; i < N; ++i) {
        a[i] = rand() % 100;
    }

    // Allocate device memory
    cudaMalloc((void **)&d_a, N * sizeof(int)); // array of N ints
    cudaMalloc((void **)&d_b, sizeof(int));     // array of 1 int (max value)

```

```
    // Copy data from host to device
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    // Max number of threads per block
    int maxThreads;
    cudaDeviceGetAttribute(&maxThreads, cudaDevAttrMaxThreadsPerBlock, 0);
    if (N < maxThreads) {
        maxThreads = N;
    }

    // Calculate number of blocks and threads
    int threadsPerBlock = maxThreads / 2; // since each thread processes 2 elements
    int blocksPerGrid = 1; // single block for this example
    size_t sharedMemSize = threadsPerBlock * sizeof(int); // shared memory size

    // Start time
    cudaEventRecord(start_cuda);

    // Repeatedly find the maximum
    int result;
    for (int run = 0; run < runs; ++run) {
        // reset output value on device
        cudaMemcpy(d_b, 0, sizeof(int), cudaMemcpyHostToDevice);

        // launch kernel (one block with N threads)
        max_reduction<<<blocksPerGrid, threadsPerBlock, sharedMemSize>>>(d_a, d_b, N);

        // Copy result back to host
        cudaMemcpy(&result, d_b, sizeof(int), cudaMemcpyDeviceToHost);
    }

    // Wait for all kernels to finish
    cudaDeviceSynchronize();

    // Stop time
    cudaEventRecord(stop_cuda);
    cudaEventSynchronize(stop_cuda);

    // Calculate average duration
    float microseconds = 0.0;
    cudaEventElapsedTime(&microseconds, start_cuda, stop_cuda);
    float avg_us = (microseconds * 1000) / runs; // convert ms to us and average

    // Print input
    std::cout << "Input array: ";
    for (int i = 0; i < N; ++i) {
        std::cout << a[i] << " ";
    }
```

```
    std::cout << std::endl;

    // Print result
    std::cout << "Max value: " << result << std::endl;

    // Print timing results
    std::cout << "Average duration: " << avg_us << " us" << std::endl;

    // Clean up
    cudaFree(d_a);
    cudaFree(d_b);
    cudaEventDestroy(start_cuda);
    cudaEventDestroy(stop_cuda);

    return 0;
}
```

Listing 3: C++ code GPU2